

mU-Code Interpreter

3단계 결과 발표

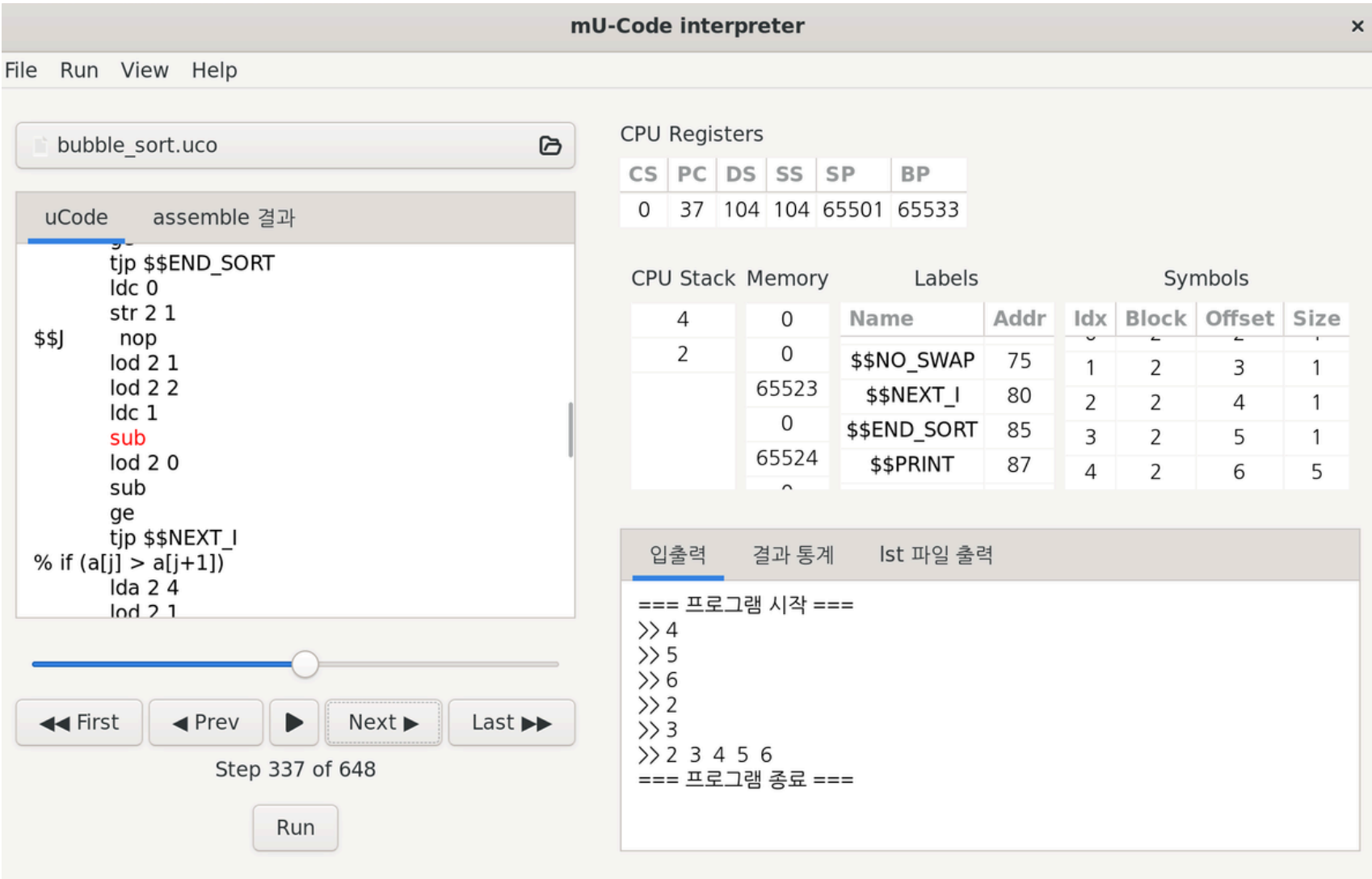
팀 UCI

컴퓨터공학과 20220744 이강석(팀장)
컴퓨터공학과 20240058 권기영

OVERALL PROJECT

프로젝트 개요

01 프로젝트 개요



GUI 실행 화면

```
===== 어셈블 시작 =====
===== 어셈블 완료 =====
===== 실행 시작 =====
1 2 4 8 9 5 10 3 6 7
[INFO] runner stopped
===== 실행 완료 =====
assets/lst_output/binary_tree_dfs.lst 에 LST 파일이 생성되었습니다.
```

CLI 실행 화면

프로젝트 결과

- mUcode를 어셈블 및 실행하는 GUI/CLI 인터프리터 개발
- 34개 명령어셋 지원
- 단계별 레지스터/메모리 시각화
- ApplImage 배포
- 예제 프로그램/벤치마크 지원

배경

- mUcode는 명령어 동작이 스택 기반이라 이해하기 어려움
- CLI는 실행 흐름/메모리 변화를 한눈에 보기 어려움

목적

- 내부 상태를 단계별로 시각화해 학습 및 분석 부담 감소
- GUI/CLI 모두 지원해 사용 환경을 확장

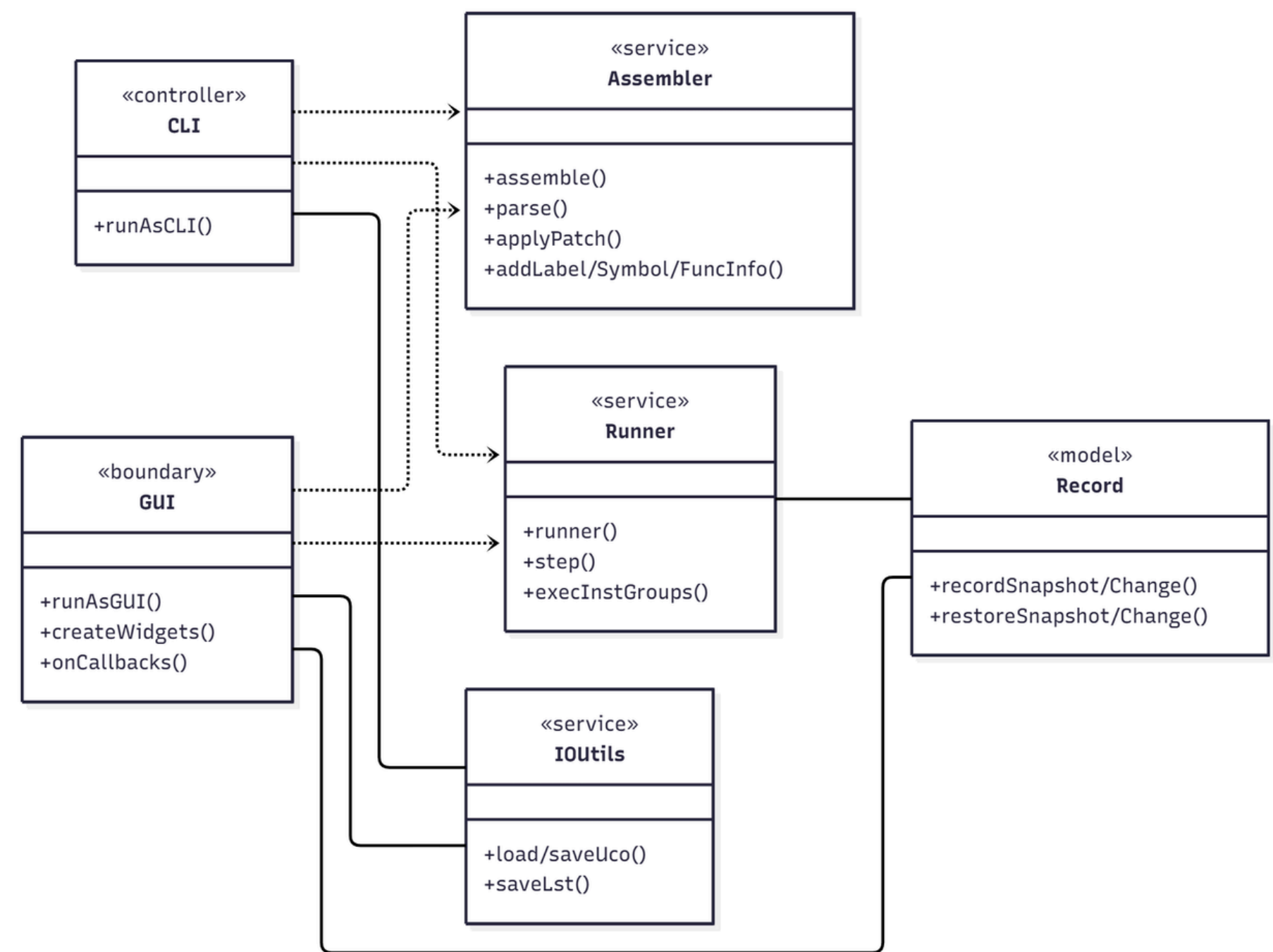
기대 효과

- 명령어 흐름 이해도 향상
- CLI 대비 가시성 증가로 학습 효율 개선
- ApplImage 배포로 리눅스의 다양한 환경에서 즉시 실행 가능

PROGRAM STRUCTURE & FEATURES

프로그램 구조 및 기능

02 프로그램 구조 및 기능



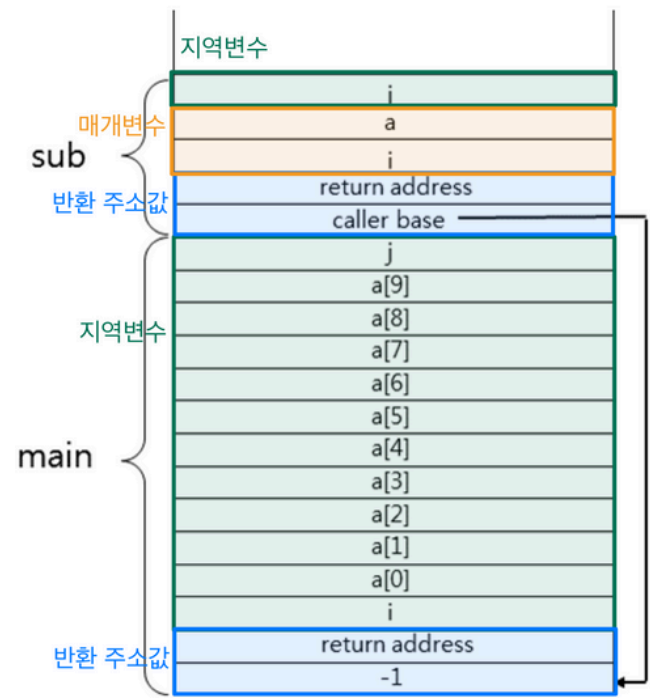
프로그램 구조 및 기능

- 인터페이스
 - CLI, GUI
- 서비스
 - Assembler: mUcode → IR, 라벨/변수/함수 정보 관리
 - Runner: IR → 결과, step 단위 명령어 실행
 - IOutils: 파일 입출력(uco, lst) 지원
- 데이터 모델
 - Record: 실행 상태 기록/복원
- 모듈 관계
 - CLI/GUI → Assembler, Runner (실행 제어)
 - Runner ↔ Record (상태 관리)
 - GUI → Record (현재 상태 복원)
 - CLI/GUI ↔ IOutils (입출력)

CHANGES AFTER INITIAL DESIGN

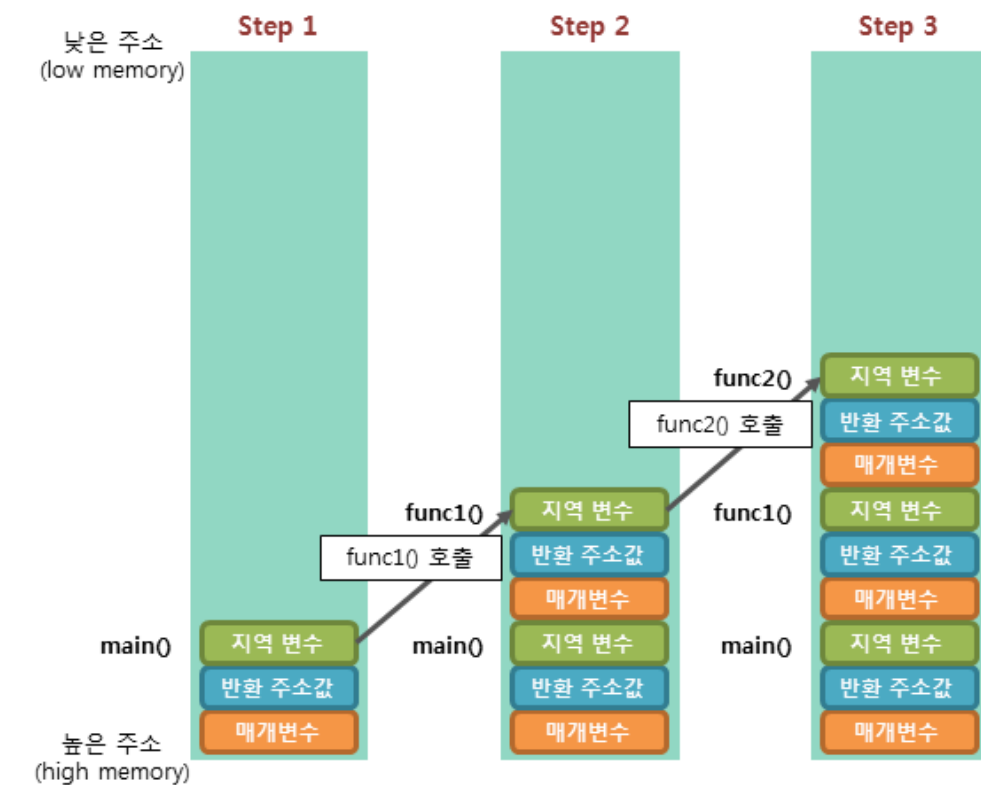
설계 이후 변경점

03 설계 이후 변경점 - 실행 엔진



<그림 5> 메모리 스택 구조

사진 출처: 코딩의 시작, TCP School



문제점1

- dup 명령어가 코드 예시에는 있지만 PDF 명세에 없음
- 전역 프레임을 종료할 때 ret 대신 end로 종료하는데 어셈블러가 이 명령어를 무시함
- 코드 흐름: ldp(함수 호출 준비) → 인자 push → call
 - ldp가 프레임 기초 공간을 먼저 만들고 그 위에 인자가 push됨
 - call은 이 기초 공간 위치를 알아야 값을 채울 수 있음
 - 하지만 인자가 차지한 공간을 몰라서 기초 공간 위치를 계산할 수 없음

변경점1

- dup 명령어 추가
- 어셈블러가 end 명령어를 ret 명령어로 매핑
- 스택 구조를 일반 아키텍처처럼 인자 먼저 쌓는 방식으로 통일

문제점2

- return 할 때 인자 개수 정보 문제로 회수가 불안정함
- 사용자가 인자 offset 값을 음수 값으로 줘야 함

변경점2

- 어셈블러가 return 시 인자 개수를 피연산자로 부여
- 어셈블러가 offset 값을 재계산해서 매핑함

03 설계 이후 변경점 - 패키지 배포

1) 패키지 관리자 기반 배포

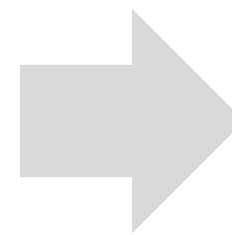
- Debian 계열 (.deb): Ubuntu 등에서 사용
 - apt로 설치/업데이트 관리
- Red Hat 계열(.rpm)
- 단점: 배포판 별로 패키지를 따로 만들어야 함

2) 소스 코드 배포

- 단점: 사용자가 직접 빌드해야 함, 의존성 문제 발생 가능

3) 범용 패키지 포맷

- AppImage, Snap, Flatpak
- 장점: 배포판 호환성 높음, 사용자 설치 간단
- 단점: 실행 속도/용량 증가



AppImage

- 장점: 단일 파일로 배포 가능
- 단점: 자동 업데이트 없음

Snap

- 장점: 자동 업데이트 지원
- 단점: 초기 설치(런타임) 필요, snap store(중앙 저장소) 의존

Flatpak

- 장점: 데스크탑 앱에 최적
- 단점: 초기 설치(런타임) 필요, 런타임 용량 큼

DEMO PROGRAM

데모 프로그램

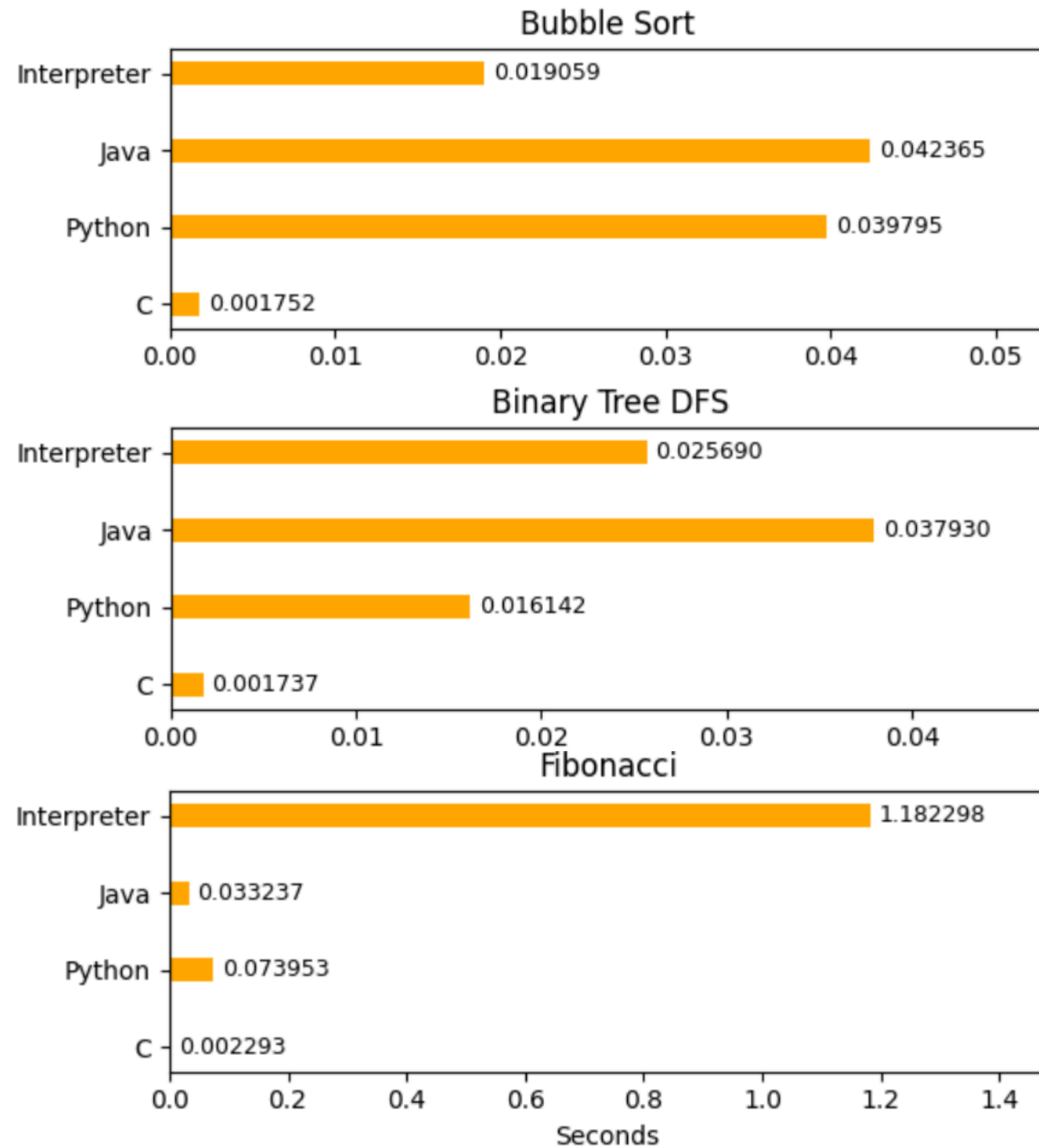
RESULT ANALYSIS

결과 분석

05 결과 분석

예제 프로그램	검증 영역	결과	비고
PDF예제	기본적인 명령어	정상 실행	SWP-3단계 PDF의 예제 활용
버블 정렬	배열, 중첩 반복문, 비교, 입출력	정상 실행	
이진 트리 탐색(DFS)	분기, 얇은 재귀 호출	정상 실행	
피보나치 수열	깊은 재귀 호출, 스택 오버플로우 처리	정상 출력 fib(21)에서 오버플로우 발생	오버플로우 테스트를 위해 전역변수, 스택 프레임의 크기를 늘린 코드 사용
최대공배수(GCD)	반복, 이항 연산	정상 실행	

05 결과 분석



실험 조건

인터프리터 설정

Snapshot 기능 : 비활성화
Change 기능 : 비활성화
LST 파일 생성 : 비활성화
기타 입출력 기능 : 제거 혹은 비활성화

실험 방법

버블 정렬 역순으로 초기화된 길이 1000의 배열 정렬
DFS 노드 수 10000개의 이진 트리 DFS 탐색
피보나치 재귀를 이용해 30번째 피보나치 수 계산
셸 스크립트에서 data +%s%N을 통해 실행시간 측정

분석

- 비교적 간단한 가상 머신 구조로 단순 계산, 반복에 유리
 - 버블 정렬, 얇은 재귀인 DFS에서 좋은 결과
- 함수 호출의 높은 비용으로 재귀 호출에서 큰 영향
 - DFS에서는 적은 영향, 깊은 재귀호출이 일어나는 피보나치에서는 큰 영향을 끼침

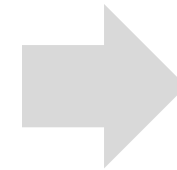
FUTURE IMPROVEMENT PLANS

향후 개선 방안

06 향후 개선 방안

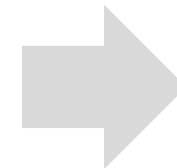
- 성능 및 메모리 최적화

Snapshot 및 Change로 인한 메모리 과다 사용



변경 내용 일정 주기마다 파일에 저장
프로그램에 따른 동적 snapshot 주기

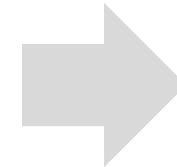
Snapshot 및 Change로 인한 실행 시간 지연



실행 스레드와 기록 스레드 분리
메모리 영역별 상태 변화 플래그 활용

- 사용성 개선

기본적인 디버그 기능만 있음



오타 수정 제안, 중단점 등의
다양한 디버깅 기능 추가

팀 UCI

컴퓨터공학과 20220744 이강석(팀장)

컴퓨터공학과 20240058 권기영